



2021년부터 백엔드 개발·운업을 담당해 왔으며, Java·Spring Boot를 주력으로 사용합니다.

B2B 리테일 교육 플랫폼과 공공·실시간 데이터 서비스의 API를 개발·운영했습니다. 상태 변경, 다중 서버의 배치 중복 실행, 구매 API 락 경합과 데이터 식별자 오류를 분석하고 서버 로직을 수정했습니다.

실무 기술

Java · Spring Boot · Spring Data JPA · QueryDSL · SQL · MySQL · MongoDB · Redis · ShedLock · Redisson · RabbitMQ · Python · Django · FastAPI · AWS · Docker · Git · JUnit · Testcontainers · REST Docs

경력

이엠캐스트(주)

2024.03 - 현재

백엔드 개발자 · 주임

4인 개발팀에서 20개 이상의 기업 고객 서비스를 제공하는 B2B 리테일 교육 플랫폼의 백엔드 개발·운업을 담당했습니다.

• 교육 플랫폼 API 개발·운영

Java·Spring Boot로 B2B 리테일 교육 플랫폼의 REST API를 설계·개발·운영했습니다. JPA·QueryDSL·MySQL로 조회·저장을 구현하고 관리자 기능 개편을 지원했습니다.

• 버전 전환·데이터 접근 계층 개선

플랫폼 2.0 → 3.0 전환의 영향 범위를 점검하고 기존 기능의 회귀를 검증했습니다. 레거시 코드와 API·DB 구조, JPA·QueryDSL 조회·저장 로직을 정리했습니다.

• 운영 오류 분석·수정·회귀 검증

운영 장애와 데이터 오류를 재현해 원인을 추적하고 API·DB 로직을 수정했습니다. 저장 시점 상태 검증, 분산락 기반 배치 제어와 구매 API 트랜잭션 분리, 복합 식별자 처리와 회귀 테스트를 다뤘습니다.

• AWS 인프라·CI/CD·배포 운영

AWS EC2·RDS·Lambda·CloudWatch·WAF 기반 배포·모니터링·운영에 참여했습니다. Docker 배포·전환 이슈에 대응하고 CI/CD 파이프라인 안정화와 코드리뷰 기반 배포 품질 관리에 참여했습니다.

주식회사 화이트스캔

2021.07 - 2024.03

백엔드·데이터 개발자 · 연구원

공공·실시간 데이터 기반 서비스의 백엔드와 데이터 처리, 배포·운업을 담당했습니다.

• 도시데이터 수집·가공·조회 API

서울 실시간 도시데이터 Open API를 수집·가공하는 파이프라인과 조회 API를 구현했습니다.

• 데이터 모델·백엔드 개발

서비스 요구사항에 맞춰 MySQL·MongoDB 스키마와 REST API를 설계하고, Spring Boot·Django·FastAPI로 데이터 조회·저장 기능을 구현했습니다.

• 예측 결과 연동·Docker 운영

시계열 예측 결과를 서비스 지표와 기능에 연동하고 관련 서비스를 Docker 컨테이너로 배포·운영했습니다.

대표 실무 문제 해결

이애플캐스트 · B2B 리테일 교육 플랫폼 개발 · 운영

저장 시점의 상태 변경으로 발생하는 데이터 오류 방지

문제 설정 변경 화면을 연 뒤 관련 작업이 시작되면, 이전 상태를 기준으로 저장하면서 진행 중인 작업에 서로 다른 처리 기준이 적용될 수 있었습니다.

수정 저장 직전 서버에서 현재 상태를 다시 확인하도록 수정했습니다. 화면 진입 후 작업이 시작됐다면 변경을 차단하고, 후속 데이터 갱신 기준도 같은 상태 조회로 통일했습니다.

검증 항목 정상 변경 · 저장 전 상태 전이 · 필수 입력 누락을 다루는 단위 테스트와 저장된 검증 기록.

구매 API의 락 경합과 트랜잭션 경계 조정

문제 동시 구매 요청에서 주문 저장 중 DB 데드락이 발생했습니다. 결제 처리를 별도 트랜잭션으로 옮기자 앞 단계의 미커밋 상태를 읽는 문제도 드러났습니다.

수정 주문 단위 분산락과 별도 트랜잭션, 락 오류의 제한적 재시도를 적용했습니다. 결제 요청 상태를 먼저 커밋하도록 호출 경계를 조정하고, 재처리 시 결제 완료 상태를 확인했습니다.

검증 항목 동시 요청 · 락 예외 · 결제 완료 재처리 테스트 정의. 저장된 초기 QA 기록의 추가 검증 항목은 별도로 관리.

다중 서버에서 동일 배치의 중복 실행 제어

문제 서버마다 Spring 스케줄러가 실행되면서 같은 배치가 중복 처리됐습니다. 작업 기록에는 조치 전 서버를 한 대로 제한한 임시 대응이 남아 있습니다.

수정 ShedLock과 Redis 기반 분산락을 스케줄러에 적용해 락을 획득한 서버가 배치를 실행하도록 변경했습니다. 배치별 락 이름과 유지 시간을 설정했습니다.

검증 항목 Redis 락 공급자 설정과 여러 스케줄러의 락 적용 커밋. 다중 서버 실행 로그와 운영 개선 수치는 별도 미확인.

복합 식별자 누락으로 발생한 삭제 · 페이지 집계 오류 수정

문제 같은 사용자의 여러 그룹 데이터를 사용자 ID만으로 구분해 삭제 대상이 누락되고, 조회 제외 조건과 페이지 집계도 실제 행 수와 어긋났습니다.

수정 사용자 ID와 그룹 유형의 복합 식별자를 조회 · 삭제 · 컬렉션 동등성에 적용했습니다. 제외 조건을 DB 쿼리로 옮기고 페이지 집계를 맞췄습니다.

검증 항목 같은 사용자 · 다른 그룹의 동시 삭제, 일부 그룹 제외와 페이지 경계를 다루는 테스트 및 저장된 검증 기록.

학력

서울과학기술대학교

영어영문학 학사 · 2018 졸업